

ConfigFabric ADDS TO REPOFABRIC

RepoFabric delivers the app. ConfigFabric makes sure it's configured right.

ConfigFabric is the desired-state layer that rides on top of RepoFabric. It assigns DSC v3 configurations to endpoints by Entra group, applies them in dependency order, reports compliance, and locks the exact app versions your configs depend on so RepoFabric never prunes a build out from under a live policy. One deployment, the same admin surface.

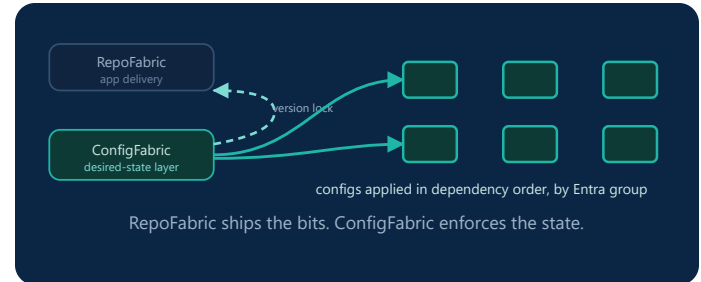
Delivery and configuration, one fabric.

Group-driven Bidirectional

Entra device groups decide which configs land where

configs lock the app versions RepoFabric must not prune*

*A published configuration that requires a specific app version registers a lock. RepoFabric consults that lock before pruning, so cleanup never breaks a configuration that is live on the fleet. The lock gate fails closed.



Group-driven assignment

Endpoints check in, the service resolves their Entra device groups, and only the configurations assigned to those groups come back, ordered and validated. Cloud-native and Hybrid devices both supported.

Dependency-aware ordering

Configs declare a phase and explicit prerequisites. The service composes a topologically-sorted manifest, so prerequisite configs apply first and a failed prerequisite halts only its dependents.

Version-lock protection

A live configuration locks the exact app versions it needs. RepoFabric checks the lock before pruning, so retention cleanup never deletes a build a deployed policy still depends on.

Same surface, one stack

Entra-gated browser portals, an Entra-attributed audit ledger, and a single docker compose deployment that shares RepoFabric's Gitea backing store when bolted on.

What it adds on top of RepoFabric

RepoFabric covers delivery. ★ marks the configuration capabilities ConfigFabric layers on.

Capability	RepoFabric alone (delivery fabric)	RepoFabric + ConfigFabric
★ Desired-state enforcement	Delivers approved app builds to endpoints over a private winget source. Configuration of the endpoint itself is out of band.	Applies DSC v3 configurations on the endpoint with <code>dsc config set</code> , in Detect (report drift) or Apply (remediate) mode, from one client engine.
★ Group-based config assignment	Endpoints point at a repo by URL; what they install is user-driven within that catalog.	A check-in resolves transitive Entra device-group membership and returns exactly the configs assigned to those groups. User-group assignment is on the roadmap.
★ Prerequisite & phase ordering	No notion of config-level prerequisites or ordered application phases.	Configs declare phases and dependency edges; the service emits a validated, acyclic, topologically-ordered manifest and halts dependents on a failed prerequisite.
★ Bidirectional version lock	Retention and cleanup can prune old app versions with no knowledge of what depends on them.	Published configs register version locks; RepoFabric consults the lock ledger before pruning and the gate fails closed, so cleanup never breaks a live policy.
★ Prerequisite resolvability checks	A config's app prerequisite can only be asserted, never verified against the catalog.	When bolted on, the resolver validates app prerequisites against RepoFabric's catalog: version satisfaction, virtual-repo coherence, and promotion stage.
Compliance reporting	Activity feed and dashboards for publish, sync, and service actions.	Adds per-device check-in, download, and compliance telemetry to Log Analytics (sparse on no-change, verbose on apply/failure) plus a client-side Windows event log.
Endpoint triggers	Endpoints pull installs at the user's discretion.	Intune Proactive Remediation, scheduled task (servers and future Arc), and interactive SYSTEM run, all over one engine, with no Windows service installed.
Deployment & auth	Browser admin, Entra sign-in, Gitea backing store, one container stack.	Reuses the same Entra sign-in, UPN-attributed audit ledger, and Gitea store; ships standalone or bolts on as a sidecar with no parallel infrastructure.